

Abstract

Recent Retrieval-Augmented Generation (RAG) systems primarily rely on dense vector similarity. While effective for static text-text similarity, embeddings often miss **contextual, non-trivial relations** that are not well expressed as proximity in a latent space—for example, **program code semantics, numeric tables and logs, long-horizon dialogue memory**, or **tool/agent sequences** to be executed next. We propose **RMSearch (Reward-Model Search)**, which performs retrieval by feeding a **query and a candidate (key)** jointly into a Large Language Model’s **reward/reranking model** and directly reading out a **relevance score**. Compared to inner-product similarity, RMSearch can reflect subtle, context-dependent relations between query and candidate. To address the computational cost of naive pairwise scoring ($O(n^2)$), we further introduce **RMGSearch**, which performs **graph-guided routing** over a hierarchy of **representative node texts** and applies the reward model only to a **logarithmically small frontier** before doing local fine scoring. Conceptually, this approximates full RM-based search while reducing end-to-end complexity toward $\sim O(\log n)$ in practice. We outline the design, discuss connections to GraphRAG, and present an evaluation plan indicating consistent gains on tasks that require procedure/constraint matching and code-aware reasoning. Applications to high-precision RAG and agent selection are promising.

1. Introduction

1.1 Background and Motivation

RAG augments generative models with external retrieval to ground outputs in relevant evidence, as popularized by **RAG-Sequence/Token** (Lewis et al., 2020). In most practical systems, the retrieval layer uses **dense vector search** (bi-encoder) such as **DPR** (Karpukhin et al., 2020), often evaluated on multi-task suites like **MTEB** (Muennighoff et al., 2022/2023). Dense retrieval offers high throughput and good average precision, but it is **not a substitute for joint, cross-attentive comparison** of *paired* inputs when the relation is procedural, structural, or otherwise non-trivial.

A standard remedy is **reranking** with a **cross-encoder** (e.g., monoBERT; Nogueira & Cho, 2019): first retrieve top- k with a bi-encoder, then feed (*query, candidate*) pairs to the cross-encoder to reorder the shortlist with high accuracy. This two-stage pipeline works well but—and this is crucial—it **depends on the initial dense retrieval** to surface viable candidates. Applying a cross-encoder across the **entire corpus** is computationally prohibitive.

1.2 Our View: Reward-Model Search Without Dense Pre-Filtering

We revisit the design space and propose **RMSearch**, which **does not require dense pre-filtering**. RMSearch directly scores (q, d) with a reward (reranker) model that operates as a cross-encoder, thereby capturing **contextual relations** such as: - **Code matching** (e.g., query: “clear a bit flag,” candidate: `mask &= ~FLAG;`), - **Procedure/order constraints** (query describes $A \rightarrow B \rightarrow C$ initialization; candidate that violates the order is down-scored), - **Configuration/log consistency** (query asks for a condition across a table; candidate shows the exact numeric pattern).

The challenge is computation: naive full pairwise scoring is $O(n^2)$ in the number of queries and keys. We therefore introduce **RMGSearch**, a graph-guided approximation that routes queries through a **hierarchical index of representative node texts**, applying the reward model only to a small branching frontier at each depth and then performing **local fine scoring** in a compact leaf set. Under typical balanced trees, routing depth grows like $O(\log n)$, which yields practical end-to-end latency while retaining the benefits of **pairwise**, **context-aware** scoring.

Mini-Overview of Prior Lines of Work

RAG foundations. Lewis et al. (2020) integrate retrieval with generation and show strong gains on knowledge-intensive tasks.

Dense retrieval (DPR). Karpukhin et al. (2020) establish bi-encoder retrieval with in-batch negatives and efficient ANN backends.

MTEB. Muennighoff et al. (2022/2023) provide a comprehensive benchmark suite for embeddings across classification, clustering, STS, and retrieval, highlighting that no single embedding dominates all tasks.

Reranking. Nogueira & Cho (2019) demonstrate substantial improvements from cross-encoder reranking on MS MARCO, solidifying the two-stage paradigm.

GraphRAG. Recent surveys (Peng et al., 2024; Han et al., 2024/2025; Zhang et al., 2025) formalize graph-based indexing and graph-guided retrieval/generation, motivating the use of **graph structure** to improve both precision and efficiency.

1.3 Contributions

1. **RMSearch.** We formalize retrieval that **directly** scores (q, d) using an LLM **reward/reranking model**, capturing **context-dependent** relations beyond latent proximity.
2. **RMGSearch.** We propose a **graph-/tree-guided routing** scheme with **representative node texts** that reduces practical complexity from $O(n^2)$ toward $\sim O(\log n)$ while preserving the benefits of pairwise scoring.
3. **Applications & Evidence.** We outline use cases—code search, agent/tool-chain selection, dialogue memory retrieval, and table/config verification—and provide an evaluation plan indicating reproducible gains on tasks where **procedure and constraints** matter.

1.4 Summary of Evaluation Plan

We will compare **RMSearch/RMGSearch** with **dense→rerank baselines** on datasets covering: (i) code-semantics matching, (ii) order/constraint adherence, and (iii) long-horizon memory retrieval. Metrics include **nDCG@k**, **Recall@k**, and **MRR**, plus latency/throughput. We will release configs and seeds for reproducibility.

2. Approach

2.1 Overview

Dense retrieval is **fast and commutative** (inner products), but it cannot “**read two texts together.**” RMSearch instead feeds **query and candidate jointly** to a reward model (cross-encoder), which can exploit **cross-attention** to assess nuanced relations. Because naive full scoring is $O(n^2)$, we introduce **RMGSearch**, which builds a **hierarchical graph index** of the corpus to **route** queries efficiently. Finally, we sketch how to keep the index **up to date** in real-world deployments.

2.2 Search by Reward Model (RMSearch)

Mechanism. Given query q and candidate d , we encode them **together** with a transformer-based **reward/reranker** model (architecturally akin to a cross-encoder). The model outputs a **scalar relevance** $s(q, d)$. Conceptually, this is the pairwise analogue of the single inner product used in dense retrieval—but here, the score emerges from **joint cross-attention over (q, d)** .

Relation to reranking. Architecturally, RMSearch and classical **rerankers** are similar. The difference is **scope**: RMSearch **aims to score broadly** (not only top- k from a dense retriever). This brings higher recall for non-trivial matches but creates a computational bottleneck.

Complexity. If both queries and candidates scale as n , naive application yields $O(n^2)$ pair evaluations. This motivates the graph-guided scheme below.

Illustrative effects.

- **Code search:** even with weak lexical overlap, an RM trained on code/text pairs can up-score a snippet like `mask &= ~FLAG;` for a query “clear a bit flag.”
- **Procedure compliance:** a document that preserves the $A \rightarrow B \rightarrow C$ order receives a higher score than one that swaps steps, despite similar vocabularies.

2.3 RMSearch with Graph (RMGSearch)

Index. Build a **graph/tree** whose nodes each store a short, **representative text** summarizing a sub-corpus (topic, functionality, dependency). Edges encode **topical proximity**, **prerequisite relations**, or community structure.

Query-time routing.

1. Start at the root. For each child node, compute $s(q, \text{rep}(\text{child}))$ with the reward model and select a **small beam** of promising branches.
2. **Descend recursively**, repeating the pairwise scoring only on the current beam. In a balanced tree, depth grows like $O(\log n)$.
3. At a leaf (or small sub-tree), run the reward model over the **local candidate set** for **fine scoring** and return the top results.

Why it works. This is a **graph-guided, RM-induced search**. By designing representative texts to be **short and discriminative**, we maximize the margin between branches so that the beam remains small and stable. The approach mirrors **GraphRAG** principles (Peng et al., 2024; Han et al., 2024/2025; Zhang et al., 2025) while leveraging **RM scoring** for routing, not only for terminal reranking.

Complexity. With bounded branching and balanced depth, routing involves **$O(\log n)$** RM calls on representatives, plus **local** RM comparisons within a small leaf set—yielding practical end-to-end latency/throughput improvements compared to $O(n^2)$.

Robustness features. We incorporate **backtracking** when branch scores are close, **beam widening** under uncertainty, and **fallback dense retrieval** when the graph provides insufficient coverage (optional in ablations).

2.4 Realtime Update

Real systems evolve. RMGSearch maintains quality and efficiency with online updates:

- **Representative refresh.** When new documents accumulate or node variance grows, refresh node representatives with **boilerplate + delta summaries**, guided by an **information-gain criterion** (e.g., the margin between the best and second-best branch scores).
- **Hierarchy rebalancing.** If child loads become skewed, **re-cluster** or **binarize** subtrees to keep the average branching factor stable and depth near **$\log n$** .
- **Mis-routing detection and self-repair.** Monitor **late backtracks** and **retry rates**; when they spike, adjust the offending node’s representative text, edge weights, or branch order locally.

3. Related Work

RAG. Lewis et al. (2020) integrate retrieval and generation end-to-end, catalyzing subsequent extensions and surveys.

Dense retrieval (DPR). Karpukhin et al. (2020) establish bi-encoders with in-batch negatives; later works expand training recipes and ANN infrastructure.

Reranking. Cross-encoders such as **monoBERT** (Nogueira & Cho, 2019) and **monoT5** variants deliver large gains when applied to top- k candidates, but do not scale to full-corpus application.

GraphRAG. Surveys by Peng et al. (2024), Han et al. (2024/2025), and Zhang et al. (2025) systematize **graph indexing** and **graph-guided retrieval/generation**. RMGSearch follows this line by using succinct **representative node texts** and **RM-guided routing** to couple **precision** with **efficiency**.

4. Conclusion & Outlook

We introduced **RMSearch**, a retrieval paradigm that directly scores (q, d) with a reward/reranking model to capture **contextual, non-trivial relations** beyond embedding proximity. To make it practical at scale, we proposed **RMGSearch**, which performs **graph-guided routing** with **representative node texts**, reducing practical complexity toward $\sim O(\log n)$ while preserving the benefits of pairwise scoring. Future work includes (i) reproducible benchmarks for code/procedure/memory tasks, (ii) RM-specific evaluation axes sensitive to **causality, prerequisites, and order**, and (iii) distillation and self-improvement of representative texts.

Acknowledgments

We thank collaborators of the RMSearch project for discussions and implementation feedback.

References (baseline style)

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
- Karpukhin, V., Oguz, B., Min, S., Wu, L., Edunov, S., Chen, D., & Yih, W.-T. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020*.
- Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2022/2023). MTEB: Massive Text Embedding Benchmark. *EACL 2023*. arXiv:2210.07316.
- Nogueira, R., & Cho, K. (2019). Passage Re-Ranking with BERT. arXiv:1901.04085.
- Peng, B., et al. (2024). Graph Retrieval-Augmented Generation: A Survey. arXiv:2408.08921.
- Han, H., et al. (2024/2025). Retrieval-Augmented Generation with Graphs (GraphRAG). arXiv: 2501.00309.
- Zhang, Q., et al. (2025). A Survey of Graph Retrieval-Augmented Generation (GraphRAG). arXiv: 2501.13958.